

✓ PASO 1: Importar librerías y cargar el Dataset

```
import pandas as pd
import numpy as np
from matplotlib import pyplot
%matplotlib inline

# Leemos el archivo que subiste a Colab
data = pd.read_csv('/content/employee_performance.csv')

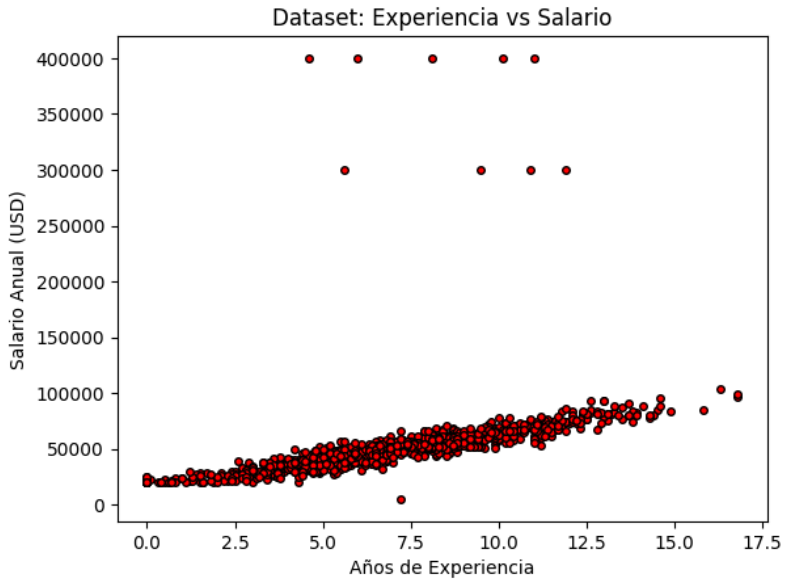
# Limpieza clave: eliminamos filas que no tengan datos en Experiencia o Salario
data = data.dropna(subset=['Experience_Years', 'Salary'])

# Extraemos nuestras variables (X = Experiencia, Y = Salario)
X_original = data['Experience_Years'].values.reshape(-1, 1)
y = data['Salary'].values
m = y.size

print("Total de datos cargados después de limpieza:", m)

# Graficamos los datos originales
pyplot.plot(X_original, y, 'ro', ms=4, mec='k')
pyplot.xlabel('Años de Experiencia')
pyplot.ylabel('Salario Anual (USD)')
pyplot.title('Dataset: Experiencia vs Salario')
pyplot.show()
```

Total de datos cargados después de limpieza: 988



leer dataset

PASO 2: Creación de características polinómicas (Grado 2)

```
# Concatenamos X con X al cuadrado (x^2) para permitir que el modelo dibuje u
X_poly = np.concatenate([X_original, X_original * X_original], axis=1)

# Función para normalizar los datos (estructurada idéntica a la del ingeniero)
def featureNormalize(X):
    X_norm = X.copy()
    # Calculamos la media
    mu = np.mean(X, axis=0)
    # Calculamos la desviación estándar
    sigma = np.std(X, axis=0)
    # Aplicamos la fórmula de normalización
    X_norm = (X - mu) / sigma
    return X_norm, mu, sigma

# Ejecutamos la función para normalizar nuestras características polinómicas
X_norm, mu, sigma = featureNormalize(X_poly)

# Añadimos la columna de unos (el término de intersección) a nuestra matriz n
X_listo = np.concatenate([np.ones((m, 1)), X_norm], axis=1)
```

```

print("¡Normalización exitosa!")
print("Matriz X lista para el entrenamiento (mostrando los primeros 5 datos):")
print(X_listo[:5])

¡Normalización exitosa!
Matriz X lista para el entrenamiento (mostrando los primeros 5 datos):
[[ 1.          -0.52457089  -0.65260438]
 [ 1.          -0.55715457  -0.67671713]
 [ 1.          1.13719686   1.1532449 ]
 [ 1.          0.84394373   0.75245193]
 [ 1.          1.39786631   1.53904886]]

```

PASO 3: Función de Costo y Descenso por el Gradiente

```

# PASO 3: Función de Costo y Descenso por el Gradiente

# 1. Función para calcular el costo (el error de nuestro modelo)
def computeCostMulti(X, y, theta):
    m = len(y)
    # Ecuación vectorizada del costo que pidió el ingeniero
    J = (1 / (2 * m)) * np.sum(np.square(np.dot(X, theta) - y))
    return J

# 2. Función del Descenso por el Gradiente (El entrenamiento)
def gradientDescentMulti(X, y, theta, alpha, num_iters):
    m = len(y)
    theta = theta.copy()
    J_history = [] # Aquí guardaremos el historial de errores para graficarlo

    for i in range(num_iters):
        # Actualización matemática de theta (ajustando los pesos)
        theta = theta - (alpha / m) * (np.dot(X, theta) - y).dot(X)
        # Guardamos el error actual para ver si vamos mejorando
        J_history.append(computeCostMulti(X, y, theta))

    return theta, J_history

# Configuramos los hiperparámetros del entrenamiento
alpha = 0.05 # Tamaño de los "pasos" que da la IA para aprender
num_iters = 2000 # Número de veces que iterará para corregir su error

# Inicializamos theta con ceros (son 3 valores porque tenemos intersección, x
theta_inicial = np.zeros(X_listo.shape[1])

print("Entrenando el modelo polinómico... ¡esto tomará un par de segundos!")

# Ejecutamos el descenso por el gradiente
theta_gd, J_history = gradientDescentMulti(X_listo, y, theta_inicial, alpha,

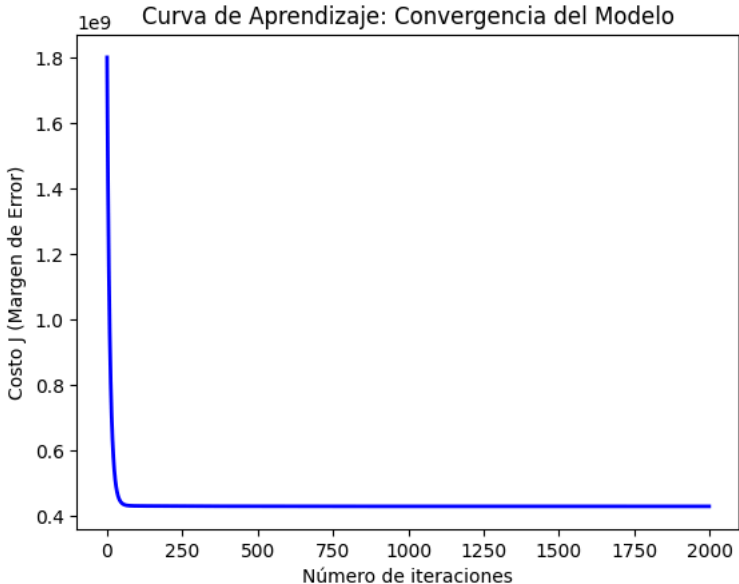
print('Los mejores valores de Theta encontrados son:', theta_gd)

```

```
# Graficamos la convergencia para verificar que el costo bajó correctamente
pyplot.plot(np.arange(len(J_history)), J_history, lw=2, color='blue')
pyplot.xlabel('Número de iteraciones')
pyplot.ylabel('Costo J (Margen de Error)')
pyplot.title('Curva de Aprendizaje: Convergencia del Modelo')
pyplot.show()
```

Entrenando el modelo polinómico... ¡esto tomará un par de segundos!

Los mejores valores de Theta encontrados son: [52977.59615385 14662.79901071



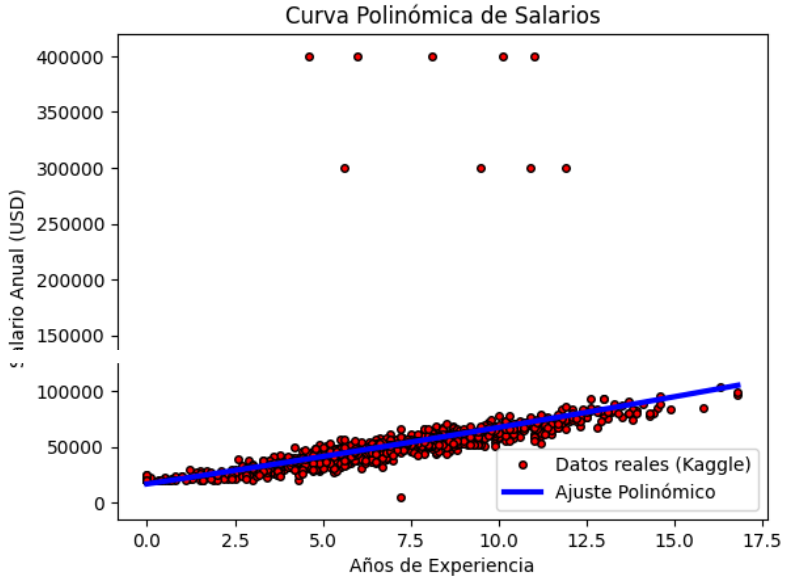
PASO 4: Predicción y visualización de la curva matemática

```
# 1. Predecir para un empleado con 10 años de experiencia
x_prueba = np.array([10, 100]) # x=10, x^2=100
x_prueba_norm = (x_prueba - mu) / sigma # Usamos los mu y sigma que guardamos
x_prueba_lista = np.concatenate([[1], x_prueba_norm])
salario_predicho = np.dot(x_prueba_lista, theta_gd)
print(f'Predicción de salario para 10 años de experiencia: ${salario_predicho}')

# 2. Graficar la curva polinómica sobre los puntos reales
# Generamos 100 puntos imaginarios entre el mínimo y máximo de experiencia pa
X_plot = np.linspace(X_original.min(), X_original.max(), 100).reshape(-1, 1)
X_plot_poly = np.concatenate([X_plot, X_plot * X_plot], axis=1) # Elevamos al
X_plot_norm = (X_plot_poly - mu) / sigma # Normalizamos
X_plot_listo = np.concatenate([np.ones((100, 1)), X_plot_norm], axis=1) # Añad
y_plot = np.dot(X_plot_listo, theta_gd) # Calculamos Y para la curva
```

```
# Dibujamos
pyplot.plot(X_original, y, 'ro', ms=4, mec='k', label='Datos reales (Kaggle)')
pyplot.plot(X_plot, y_plot, 'b-', lw=3, label='Ajuste Polinómico')
pyplot.xlabel('Años de Experiencia')
pyplot.ylabel('Salario Anual (USD)')
pyplot.title('Curva Polinómica de Salarios')
pyplot.legend()
pyplot.show()
```

Predicción de salario para 10 años de experiencia: \$67419.92



PASO 5: Cálculo exacto con la Ecuación de la Normal (Sin iteraciones)

```
print("Calculando parámetros con la Ecuación de la Normal...")

# Añadimos los unos directamente al X polinómico ORIGINAL (sin normalizar)
X_normal = np.concatenate([np.ones((m, 1)), X_poly], axis=1)

def normalEqn(X, y):
    # Fórmula matricial: theta = (X^T * X)^-1 * X^T * y
    theta = np.dot(np.dot(np.linalg.inv(np.dot(X.T, X)), X.T), y)
    return theta

# Calculamos theta
theta_normal = normalEqn(X_normal, y)
print('Theta calculado por la Ecuación de la Normal:', theta_normal)

# Hacemos la misma predicción para 10 años de experiencia
```

```
# Como no normalizamos, usamos los datos crudos [1, x, x^2]
x_prueba_normal = np.array([1, 10, 100])
salario_predicho_normal = np.dot(x_prueba_normal, theta_normal)
print(f'Predicción Normal (10 años): ${salario_predicho_normal:.2f}')
```